



École nationale des ponts et chaussées

Projet de département IMI - 2025-2026

Antoine CHOSSON
Clara LIMA-GOLDENBERG
Youssef SAHRAOUI
Yassine KOUAS

Sous la direction de : Julien REYGNER, Adrien TOUBOUL
Entreprise : Advansight

Réduction du coût de calcul et de mémoire pour les
LLMs locaux

Approche Kernelisée du calcul d'attention

Point d'avancement, Avril 2026

1 Présentation du projet

1.1 Contexte

Ce Projet s'inscrit dans une démarche applicative et analytique d'approches kernelisées afin de réduire les coûts de calculs de LLMs. Les approches étudiées sont scientifiques et relèvent de papiers de recherche en machine learning. Le "toy modèle" sur lequel nous expérimentons est OpenSource et largement utilisé par le grand public.

1.2 Problématique, objectifs

Le mécanisme de calcul d'attention des transformers présente la contrainte d'être quadratique en temps et mémoire par rapport à la longueur de la séquence traitée. Le but de ce projet est d'étudier une approche kernelisée (*Performer FAVOR+ Attention*) de ce calcul d'attention rendant les calculs linéaires. Une étude à la fois théorique sur la variance, la précision et la convergence de l'approximation ; ainsi qu'expérimentale en remplaçant l'attention dans un modèle open source par son architecture kernelisée.

1.3 État de l'art

Depuis 2020, la communauté du Machine Learning s'intéresse aux méthodes kernelisées afin de rendre le calcul de l'attention linéaire [**transformersareRNNs**]. En particulier, le théorème de Mercer (voir annexe) permet d'exprimer un noyau sous la forme d'un produit scalaire entre fonctions de caractéristiques, $k(x, y) = \langle \phi(x), \phi(y) \rangle$. Ce cadre théorique est exploité par Choromanski et al. dans *Rethinking Attention with Performers* [**rethinkingattentionwithperformers**], en approchant le noyau softmax par une factorisation de type $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$.

Cette reformulation permet d'éviter le calcul explicite de la matrice d'attention, réduisant la complexité de $\mathcal{O}(N^2)$ à $\mathcal{O}(N)$. Afin de garantir la positivité et la stabilité numérique de l'approximation, les auteurs introduisent la méthode FAVOR+ (*Fast Attention Via positive Orthogonal Random features*), qui repose sur une approximation stochastique du noyau à l'aide de projections aléatoires.

2 Formalisation du problème

- Les entrées principales sont des tenseurs PyTorch de type vecteurs réels : les requêtes Q, les clés K et les valeurs V de dimension [B,H,N,D], où B est la taille de batch, H le nombre de têtes, N la longueur de séquence et D la dimension des embeddings.
- Les sorties sont également des tenseurs vectoriels de même structure, correspondant aux représentations contextualisées après attention.
- Les données manipulées sont des séquences de tokens, transformées en embeddings numériques.
- Les valeurs typiques pour N sont de l'ordre de $10^3 - 10^4$, et pour D de l'ordre de 64 ou 128. Le nombre de features aléatoires lui va de 64 à 512.
- Les unités sont abstraites (espace vectoriel), sans dimension physique.
- La qualité des résultats est évaluée à l'aide de critères probabilistes, en comparant les distributions du modèle classique et du modèle performer. Nous utilisons la divergence de Kullback-Leibler (KL) entre les distributions d'attention exacte et approchée, ainsi que des métriques comme la probabilité du token top-1 ou l'erreur moyenne.

3 Avancement du projet

3.1 Organisation du groupe

- **Antoine** : Implémentation PyTorch de l'attention Performer et remplacement dans le modèle TinyLlama 1.1B. Tests et analyses de complexité et convergence en Python.
- **Clara** : Analyser le fonctionnement et les limites du Performer, explorer des alternatives pour l'approximation du noyau exponentiel et réaliser une étude spectrale des différentes méthodes pour identifier les causes de perte de performance.
- **Yassine et Youssef** : Implémentation de la variante FAVOR# et benchmark comparatif avec FAVOR+. Étude via la métrique de la divergence de Jensen–Shannon.

3.2 Contact avec le tuteur

Nous menons un point visio hebdomadaire avec notre tuteur accompagné de slides ou trace écrite pour présenter notre avancement. Pour le code, un repo GitHub regroupe l'ensemble des tests et l'implémentation. Pour la théorie nous avons un drive partagé regroupant les papiers de recherche ainsi qu'un document résumant le contenu et ce que nous retenons de chacun.

3.3 Travail effectivement réalisé, premiers résultats

Nous avons implémenté une class python d'attention Performer avec Orthogonal Random Sampling de features (FAVOR+). Nous avons adapté cette classe générique à l'architecture native du modèle OpenSource TinyLlama 1.1B afin de pouvoir remplacer une ou plusieurs de ses têtes d'attention par des Performer et de benchmark ses performances (vitesse et qualité de génération). Ces premiers résultats ont mis en évidence l'avantage computationnel (jusqu'à 3x plus rapide) du Performer pour les grandes dimensions (voir résultats ci dessous).

Prefill step H=32 D=64				
N	Classic (ms)	Perf M=128	Perf M=256	speedup
256	0.26	1.05	1.72	0.24x
512	0.55	2.07	3.28	0.27x
1024	2.03	4.09	5.15	0.50x
2048	7.99	7.49	9.85	1.07x
4096	39.75	12.95	19.78	3.07x

FIGURE 1 – Benchmark des vitesses de calcul : Attention standard vs Performer – Valeurs en ms

Nous avons également pu comparer la qualité de génération en réponse à un prompt. En remplaçant 4/32 têtes d'attention, le modèle est toujours proche du modèle classique softmax (ce qui valide notre implémentation) et se dégrade en remplaçant davantage de têtes. Notre objectif sera après fine-tuning d'avoir des niveaux de performances équivalents pour 32/32 têtes de type Performer.

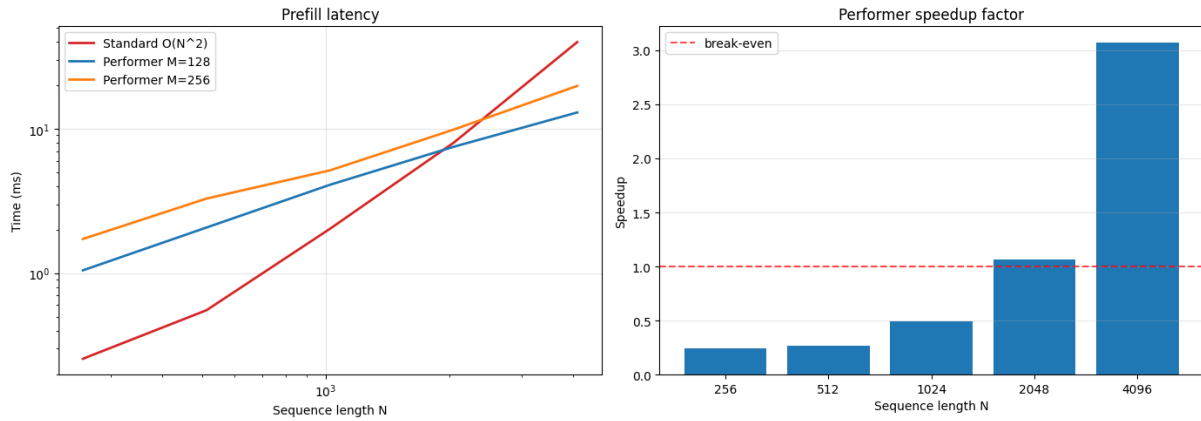


FIGURE 2 – Benchmark des vitesses de calcul : Attention standard vs Performer – Plots

Reponse to prompt : How do I get a good night's sleep?

Step	Classic.....	Performer.....	p(cls)	p_perf(cls)	KL
1	To	To	29.2%	28.6%	0.56
2	get	get	96.3%	85.5%	0.07
3	a	a	95.2%	94.2%	0.00
4	good	good	99.7%	98.3%	0.01
5	night	night	99.9%	99.8%	0.00
6	,	,	99.9%	99.6%	0.00
7	s	s	100.0%	94.4%	0.06
8	sleep	sleep	99.9%	99.7%	0.00
9	,	,	98.6%	98.1%	0.01
10	here	you	24.1%	1.5%	0.70
11	are	are	99.4%	96.0%	0.02
12	some	some	89.6%	82.8%	0.02
13	tips	tips	90.8%	80.8%	0.05
14	:	:	66.1%	61.0%	0.03
15	\n	\n	99.9%	99.3%	0.00
16	\n	\n	98.0%	97.8%	0.00
17	1	1	99.9%	99.6%	0.00
18	.	.	100.0%	99.8%	0.00
19	Create	Est	50.7%	18.4%	0.37
20	a	a	98.7%	97.9%	0.00
21	relax	relax	32.7%	33.4%	0.12
22	ing	ing	98.2%	83.7%	0.12
23	sleep	sleep	43.7%	42.6%	0.07
24	environment	environment	96.2%	51.3%	0.52
25	:	.	93.4%	35.9%	0.76
26	Make	Make	43.3%	12.2%	0.41
27	sure	sure	59.5%	70.1%	0.04
28	your	your	87.0%	67.2%	0.11
29	bed	bed	71.2%	72.5%	0.02
30	room	room	95.7%	95.8%	0.02

FIGURE 3 – Comparaison token par token de la réponse à un prompt : classique vs hybride 4/32 performer

D'un point de vue théorique, plusieurs travaux analysant les approximations de la fonction softmax (notamment au sein des modèles de type Performers) mettent en évidence une dégradation de la pertinence des prédictions. Ce phénomène se traduit par une incapacité croissante du modèle à distinguer le bruit contextuel des informations cruciales, ou à réaliser des tâches de type « needle in a haystack ».

Pour expliciter ce mécanisme, nous étudions le spectre du noyau exponentiel (dont la défi-

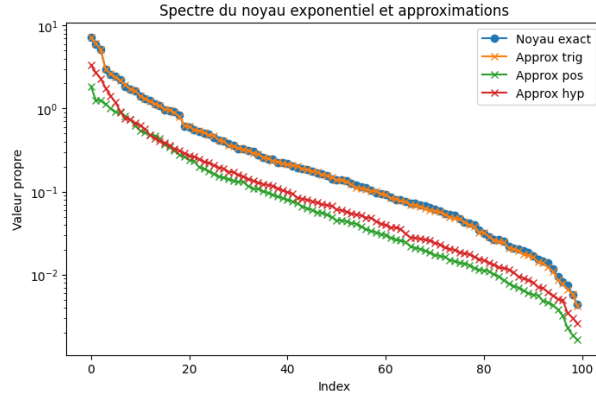


FIGURE 4 – Valeurs propres approchées du noyau exponentiel et des approximations proposés dans le papier *Performer*

nition précise est fournie en annexe) ainsi que de ses approximations. On observe une perte de précision qui affecte principalement les petites valeurs propres.

Parallèlement, diverses méthodes ont été développées pour approcher ce noyau exponentiel. Nous explorons ici des alternatives d'approximation avec l'objectif suivant : une fois identifiée la cause profonde des baisses de performance dans les architectures Transformer, nous pourrions privilégier une méthode capable de remédier à ces lacunes spécifiques.

3.4 Difficultés rencontrées (ou non)

Tester et faire tourner les modèles en local requiert trop de puissance de calcul et la génération de quelques tokens peut prendre plusieurs minutes ce qui retarde les phases de debug et de tests. Nous utilisons pour l'instant les GPUs de colab, nous aurons peut être besoin de ressources supplémentaires pour fine tuner nos modèles modifiés.

La quantité de publications traitant de la performance des LLM est considérable. Discerner les contributions utiles et vérifiées constitue un véritable défi, rendant l'identification de travaux spécifiques d'autant plus complexe. C'est vrai par exemple pour le sujet d'analyse spectrale appliquée aux Transformers.

3.5 Suite du projet, prochaines étapes (court terme)

Notre implémentation étant validée mathématiquement, il faut maintenant fine tuner le modèle pour augmenter la qualité de génération. Les vecteurs Q et K doivent être adaptés à l'attention Performer, ils sont pour l'instant entraînés sur une attention softmax, rendant la génération avec performer incohérente. Nous allons utiliser une méthode de fine tuning léger et de bas niveau de type LoRA.

Nous envisageons également de tester nos résultats théoriques avec ces modèles modifiés.

Dans la section consacrée à l'analyse spectrale, nous cherchons à exprimer l'erreur observée en fonction des valeurs propres du noyau exponentiel et de ses approximations. L'objectif est également de comprendre s'il est possible d'établir une corrélation entre ces dernières. Enfin, en envisageant de tester des approximations non présentées dans l'article original des Performers, nous analyserons également leurs spectres de valeurs propres.

4 Analyse critique

4.1 Positionnement des résultats

L’approche kernelisée sur laquelle nous nous concentrons s’inscrit bien dans l’esprit du sujet. Elle est particulièrement pertinente car elle présente un niveau de complexité modéré, ce qui permet une implémentation accessible en Python avec PyTorch. Par ailleurs, elle repose sur une base théorique riche et nous ouvre la voie à de nombreux résultats annexes que nous pouvons analyser, expérimenter et valider empiriquement.

4.2 Apports du groupe

Notre contribution en tant que groupe consiste à tester la méthode Performer sur des modèles open source. Bien que l’approche kernelisée soit relativement bien documentée, elle n’a quasiment jamais été implémentée sur des LLMs grand public. Par ailleurs, nous abordons cette approche avec une composante très mathématique, en nous appuyant non seulement sur des papiers de ML, mais aussi sur des références en mathématiques théoriques. Cela nous permet d’étudier des propriétés comme la convergence et la variance, que nous chercherons ensuite à valider expérimentalement.

A Annexes

A.1 Théorème de Mercer (1909)

D'après le théorème de Mercer, tout noyau continu, symétrique et défini positif $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ sur un espace compact admet une décomposition spectrale de la forme :

$$k(x, y) = \sum_{m=1}^{\infty} \lambda_m \psi_m(x) \psi_m(y),$$

où (λ_m) sont des valeurs propres positives et (ψ_m) une base orthonormée de $L^2(\mathcal{X})$. Cela implique l'existence d'une application ϕ telle que :

$$k(x, y) = \langle \phi(x), \phi(y) \rangle.$$

A.2 Spectre des noyaux symétriques

Soit k un noyau défini positif. Soit μ une mesure de probabilité sur $\mathbb{R}^d$ et $L_2(\mu)$ l'espace des fonctions de carré intégrable par rapport à μ . On définit l'opérateur intégral T_k : pour tout $f \in L_2(\mu)$ et $x \in \mathbb{R}^d$,

$$T_k f(x) = \int_{\mathbb{R}^d} k(x, y) f(y) d\mu(y)$$

Le spectre de k est l'ensemble des valeurs propres $(\lambda_m)_{m \geq 0}$ de T_k , c'est-à-dire les réels tels qu'il existe $\phi_m \in L_2(\mu)$ non nul vérifiant :

$$T_k \phi_m = \lambda_m \phi_m$$

A.3 Exemple de code

Implémentation en python de la méthode *Orthogonal Random Feature Sampling* caractéristique de la version FAVOR+ du Performer

```
1 def _sample_orf(head_dim, num_features, device=None):
2     if device is None:
3         device = torch.device("cpu")
4     blocks = []
5     while len(blocks) * head_dim < num_features:
6         G = torch.randn(head_dim, head_dim, device=device, dtype=torch.float32)
7         Q, _ = torch.linalg.qr(G)
8         norms = torch.randn(head_dim, head_dim, device=device).norm(dim=1)
9         blocks.append(torch.diag(norms) @ Q.T)
10    return torch.cat(blocks, dim=0)[:num_features]
```

Classe définissant l'architecture générale d'une tête d'attention type Performer. Adaptée par la suite à l'architecture native du modèle TinyLlama1.1B pour remplacement.

```
1 class PerformerAttention(nn.Module):
2
3     def __init__(self, dim, num_heads, head_dim, num_features):
4         super().__init__()
5         self.dim = dim
6         self.num_heads = num_heads
7         self.head_dim = head_dim
8         self.num_features = num_features
9
```

```

10     self.register_buffer("omega", _sample_orf(head_dim, num_features))
11
12     inner_dim = num_heads * head_dim
13     self.q_proj = nn.Linear(dim, inner_dim, bias=False)
14     self.k_proj = nn.Linear(dim, inner_dim, bias=False)
15     self.v_proj = nn.Linear(dim, inner_dim, bias=False)
16     self.out_proj = nn.Linear(inner_dim, dim)
17
18     def phi(self, x, is_query=True):
19         return _phi(x, self.omega, self.num_features, is_query)
20
21     def forward(self, x):
22         B, N, _ = x.shape
23         q = self.q_proj(x).view(B, N, self.num_heads, self.head_dim).transpose(1, 2)
24         k = self.k_proj(x).view(B, N, self.num_heads, self.head_dim).transpose(1, 2)
25         v = self.v_proj(x).view(B, N, self.num_heads, self.head_dim).transpose(1, 2)
26
27         scale = self.head_dim ** -0.25
28         phi_q = self.phi(q * scale, is_query=True)
29         phi_k = self.phi(k * scale, is_query=False)
30
31         kv_cumsum = torch.einsum("bhnnm,bnd->bhnmd", phi_k, v).cumsum(dim=2)
32         k_cumsum = phi_k.cumsum(dim=2)
33         out = torch.einsum("bnm,bnmd->bnd", phi_q, kv_cumsum)
34         z = 1 / (torch.einsum("bnm,bnm->bn", phi_q, k_cumsum) + 1e-6)
35         out = out * z.unsqueeze(-1)
36
37         out = out.transpose(1, 2).contiguous().view(B, N, -1)
38         return self.out_proj(out)

```

Références

- [1] Rethinking Attention With Performers – Choromanski, Krzysztof and others, ICLR 2021
- [2] Transformers are RNNs : Fast Autoregressive Transformers with Linear Attention – Katharopoulos, Angelos and others, ICML 2020